

MDP (Model Data Protocol) Whitepaper

Version: 1.0

Date: November 7, 2025

1. Introduction

The Model Data Protocol (MDP) is an open standard designed to unify the way large language models and AI agents persist, share and retrieve conversation context across disparate providers. Today, each vendor offers their own conversation storage and tooling layer, making it difficult to build applications that need to work seamlessly with multiple AI models such as ChatGPT, Claude, Gemini or Azure-OpenAI. MDP solves this fragmentation by providing a vendor-neutral memory layer and a portable conversation format. Once stored, any provider adapter can read and write to the same data store without reinventing the wheel.

2. Goals and Principles

- Vendor neutrality: MDP does not depend on a particular provider or model architecture. Interactions with OpenAI, Anthropic, Google and Azure are all represented in the same canonical format.
- Portability and interoperability: Conversations are stored as JSONL exports and are portable across systems. Developers can import or export history without being locked into a single vendor.
- Unified memory layer: A shared memory service with semantic search and hybrid retrieval ensures that models have access to relevant context regardless of which provider generated it.

- Security and governance: Strong controls over access, encryption, PII tagging, auditing and cost tracking are first-class concerns rather than afterthoughts.

3. Architecture Overview

At the heart of MDP lies the server, which exposes REST and WebSocket APIs. Clients send messages to the server, which stores them in a relational database and passes them along to the appropriate provider adapter. Each adapter translates MDP messages into provider-specific payloads, streams the response back and records token usage and cost in Canadian dollars (CAD). The memory service selects relevant memory entries using hybrid keyword and vector search to augment each request. Files and tool outputs are stored as artifacts alongside messages. A tenant/space hierarchy enforces logical boundaries between users, teams and projects.

4. Protocol Specification

MDP defines several core entities:

- Tenant: Represents an organisation or user owning data. Tenants contain spaces.
- Space: A logical boundary (personal, team, project, organisational) that groups conversations and memory. Each space has an ID, label and metadata.
- Conversation: A thread of messages. Conversations track creation and update times, titles and tags.
- Message: An atomic unit in a conversation with a role (user, assistant, system or tool) and content blocks (text, image, file reference or tool call). Messages record token counts and cost.
- MemoryEntry: A standalone fact, preference, document or embedding with a scope (user,

team, global), PII flags, expiry and ACL. Memory entries can reference the source conversation, message, file or an external URL.

- ProviderRef: Captures the provider name, model and request identifier for auditing and cost calculation.

- ACL: Defines fine-grained access control policies on memory entries or spaces, supporting allow and deny rules.

5. Storage and Persistence

MDP uses PostgreSQL with the pgvector extension for persistent storage. Conversation and message metadata live in relational tables. Embeddings are stored in a vector column enabling similarity search across memory and messages. Files are stored in object storage (e.g., S3 or Azure Blob), while metadata about artifacts resides in the database. For local development, SQLite with an in-memory FAISS index can be used as a lightweight alternative.

6. Provider Adapters

Adapters abstract away provider-specific APIs. MDP includes first-party adapters for OpenAI, Anthropic, Google (Gemini) and Azure-OpenAI. Each adapter implements a unified interface to send chat requests and receive streaming responses. Additional adapters can be added without modifying core logic. Adapters normalise token usage and cost into CAD using a configurable pricing table, ensuring that applications can compare providers fairly.

7. Cross-Model Memory and Retrieval

One of MDP's distinguishing features is its ability to supply cross-model memory. When a

message is sent, the server calls the memory service to retrieve relevant facts and documents based on keywords and embeddings. These memory entries are then injected into the request context before hitting the provider. Models can also write new memories: for example, an extraction tool can summarise key facts from a conversation and store them as long-lived memories. Memories support expiry times (TTL) and scopes (user, team, global) so that sensitive or obsolete data can be purged automatically.

8. Security and Governance

Security is paramount. MDP implements authentication and authorisation via JWT and space-level RBAC. All data at rest is encrypted; sensitive PII in memory entries is masked. Audit events record who accessed or modified what, when and why. Cost tracking is mandatory: every message stores prompt and completion tokens and the normalised cost in CAD. Administrators can enforce policies such as which providers are allowed, maximum token limits and tool call restrictions. A separate reverse proxy or gateway can handle perimeter authentication and rate limiting.

9. Use Cases

MDP enables a wide range of applications:

- Multi-model assistants: Build an AI agent that can switch between models mid-conversation while sharing the same memory.
- Enterprise knowledge bases: Index internal documents once and make them available to all providers with consistent retrieval.
- Compliance and audit: Maintain a complete record of conversations, tool calls and memory reads/writes for legal or regulatory purposes.
- Cost analysis: Compare the operational cost of different providers on the same workload.

- Collaboration: Share conversations and memories across teams without duplicating data.

10. Implementation Guide

The reference implementation is organised as a monorepo with pnpm workspaces:

mdp/

apps/

mdp-server/ – Fastify server exposing REST and WebSocket endpoints

mdp-console/ – (Optional) Next.js admin console

mdp-gateway/ – Reverse proxy for authentication and rate limiting

packages/

mdp-core/ – Protocol types and interfaces

mdp-schema/ – Prisma schema and migrations

mdp-storage/ – Storage drivers (Postgres/SQLite/Vector)

mdp-adapters/ – Provider adapters (OpenAI, Anthropic, Google, Azure)

mdp-memory/ – Memory service with hybrid retrieval

mdp-tools/ – Tool registry and contract definitions

mdp-client-js/ – JavaScript client SDK

mdp-spec/ – Protocol documentation (OpenAPI)

sdk/

python/ – Python SDK

dotnet/ – .NET SDK

examples/

quickstart-node/

infra/

docker/ – Docker Compose with Postgres and pgvector

k8s/ – Helm charts for Kubernetes

terraform/ – IaC for cloud infrastructure

To get started locally:

1. Clone the repository and install dependencies with ``pnpm install``.
2. Start Postgres and the vector extension using ``docker compose up``.
3. Run ``pnpm prisma migrate dev`` to create the database schema.
4. Set your provider API keys in a ``.env`` file.

5. Launch the server with ``pnpm --filter mdp-server dev``.

6. Use the CLI (``mdp chat ...``) or your own code to interact with the API.

11. Conclusion and Future Work

MDP lays the foundation for interoperable AI agents by offering a shared, secure and extensible memory layer. As more providers and tools emerge, a common protocol becomes increasingly important to avoid data silos and vendor lock-in. Future enhancements may include real-time collaboration, fine-grained data classification, richer event streams, a graphical console and contributions from the community. We invite developers and organisations to adopt, extend and contribute to MDP.